

Full-Stack · AI/ML · Salesforce · System Design

[Q&A Pairs](#)
[Concept Notes](#)

[Topic Sections](#)
[Soft-Skill Answers](#)

- 1 Introduction & Career Objective
- 2 Java & Object-Oriented Programming
- 3 Spring Boot & REST APIs
- 4 Databases — PostgreSQL & MySQL
- 5 React.js & Frontend Development
- 6 AI / ML — XGBoost, LSTM, NLP, LLMs
- 7 Salesforce — Apex, LWC, SOQL & Flows
- 8 Data Structures & Algorithms
- 9 Operating Systems & Computer Networks
- 10 Tools & Platforms — Git, Docker, Vercel
- 11 Project Deep Dive — VigilAI MedLink v2.0
- 12 Certifications & Achievements
- ★ Soft Skills — Most Asked Questions

1

Introduction & Career Objective

Q1

Tell me about yourself.

✓ Answer

I'm Omkar, a second-year Computer Science student at REVA University (CGPA 8.6). I'm a Full-Stack Developer skilled in React.js, Spring Boot, PostgreSQL, and Salesforce. I've built real-world projects like VigilAI MedLink v2.0 — a healthcare triage platform with AI risk prediction — and a Job Recruitment Portal on Salesforce. I'm passionate about combining backend engineering with AI/ML to solve practical problems. I also won the Datathon 2025 at REVA University.

■ **Tip:** Keep this to 90 seconds. Memorize a version tailored to each company's domain (healthcare, fintech, etc.).

Q2

What is your career objective?

✓ Answer

My goal is to become a well-rounded Software Engineer who can architect scalable systems and integrate AI/ML to solve real-world problems. In the short term, I want to gain industry experience in full-stack development or backend engineering. Long-term, I aim to lead engineering teams building intelligent, data-driven applications.

■ **Tip:** Tailor this to the company's domain before each interview.

Q3

Why did you choose Computer Science?

✓ Answer

I've always been fascinated by how software can solve complex problems at scale. My interest in building things that people use — from web apps to AI models — led me to choose CSE. The ability to combine logic, creativity, and technology in one discipline is what keeps me deeply motivated.

Q4

Where do you see yourself in 5 years?

✓ Answer

In 5 years, I see myself as a Senior Software Engineer or Tech Lead, having shipped production-grade systems at scale. I aim to have deep expertise in distributed systems, cloud-native architectures, and applied AI. I also want to mentor junior developers and contribute to open-source projects.

2 Java & Object-Oriented Programming

Q5 What are the four pillars of OOP? Explain with examples.

✓ Answer

- Encapsulation: Bundling data and methods together, hiding internal state. Example: private fields with public getters/setters.
- Abstraction: Hiding implementation details and exposing only necessary functionality. Example: abstract class or interface defining a contract.
- Inheritance: A child class acquires properties of a parent class. Example: Dog extends Animal.
- Polymorphism: Same method behaves differently based on context. Example: method overriding (runtime) and method overloading (compile-time).

■ *Tip: Use your VigilAI or EMS project as a concrete code example.*

Q6 What is the difference between abstract class and interface?

✓ Answer

Abstract class: Can have both abstract and concrete methods, instance variables, constructors; supports single inheritance.

Interface: Before Java 8, only abstract methods; from Java 8+, can have default and static methods; supports multiple inheritance.

Use abstract class when sharing code among closely related classes. Use interface when unrelated classes implement the same contract.

Q7 What is the difference between `==` and `.equals()` in Java?

✓ Answer

`==` checks reference equality — whether two variables point to the same object in memory.

`.equals()` checks logical/value equality — whether two objects are semantically the same.

Example: Two String objects with value 'hello' will have `==` return false (different references) but `.equals()` return true.

Q8

Explain the static keyword in Java.**✓ Answer**

static makes a member belong to the class rather than to any instance.

- Static variable: Shared across all instances.
- Static method: Can be called without creating an object (e.g., `Math.sqrt()`).
- Static block: Runs once when the class is loaded, used for static initialization.
- Static nested class: Doesn't need an outer class instance.

Q9

What is the difference between ArrayList and LinkedList?**✓ Answer**

ArrayList: Backed by a dynamic array; $O(1)$ random access; $O(n)$ insert/delete in the middle; better for read-heavy workloads.

LinkedList: Doubly-linked list; $O(n)$ random access; $O(1)$ insert/delete at known positions; better for frequent insertions/deletions at ends.

Q10

What is Exception Handling in Java? Difference between checked and unchecked exceptions?**✓ Answer**

Exception handling is managed via try-catch-finally blocks.

Checked exceptions: Checked at compile time; must be caught or declared (e.g., `IOException`, `SQLException`).

Unchecked exceptions: Runtime exceptions; not enforced by the compiler (e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`).

In Spring Boot, use `@ControllerAdvice` + `@ExceptionHandler` for global exception handling.

■ **Tip:** Mention how you used exception handling in your REST APIs.

Q11

What is the Java Collections Framework?**✓ Answer**

A unified architecture for storing and manipulating groups of objects.

Key interfaces: List (ArrayList, LinkedList), Set (HashSet, TreeSet), Map (HashMap, TreeMap), Queue (PriorityQueue).

- HashMap: $O(1)$ average for get/put; uses `key.hashCode()`.
- TreeMap: Sorted order; $O(\log n)$.
- HashSet: No duplicates; backed by HashMap.

Q12

What are Java Streams? How have you used them?**✓ Answer**

Streams (Java 8+) allow functional-style operations on collections: filter, map, reduce, collect.

Example: `employees.stream().filter(e -> e.getSalary() > 50000).collect(Collectors.toList())`.

In my Employee Management System, I used streams to filter and sort employee records before returning them from REST endpoints.

Q13

What is multithreading? Difference between Thread and Runnable?**✓ Answer**

Multithreading allows concurrent execution of two or more threads.

Thread: Extending Thread class; less flexible since Java doesn't support multiple class inheritance.

Runnable: Implementing Runnable interface; preferred because it separates task from execution, allows the class to extend another class, and works well with ExecutorService.

Q14

What is the difference between HashMap and ConcurrentHashMap?**✓ Answer**

HashMap: Not thread-safe; suitable for single-threaded use.

ConcurrentHashMap: Thread-safe; uses segment-level locking (Java 7) or CAS operations (Java 8+); allows concurrent reads and writes without blocking the entire map.

3 Spring Boot & REST APIs

Q15

What is Spring Boot and how is it different from Spring Framework?**✓ Answer**

Spring Framework is a comprehensive framework requiring manual XML/Java configuration.

Spring Boot is an opinionated, convention-over-configuration layer on top of Spring that:

- Auto-configures beans based on classpath.
- Provides embedded servers (Tomcat, Jetty).
- Uses starter dependencies to simplify setup.
- Eliminates boilerplate configuration.

This allowed me to build VigilAI MedLink and the Employee Management System rapidly without complex setup.

Q16

Explain the flow of a REST API request in Spring Boot.**✓ Answer**

1. Client sends HTTP request.
2. DispatcherServlet receives and routes it.
3. @RestController method is matched via @RequestMapping.
4. Service layer is called for business logic.
5. Repository (JPA/JDBC) interacts with the database.
6. Response is serialized to JSON and returned.

In my EMS project, I implemented all CRUD operations following this exact flow.

Q17

What is Dependency Injection? How does Spring implement it?**✓ Answer**

DI is a design pattern where objects receive their dependencies from an external source rather than creating them.

Spring implements DI via:

- Constructor injection (preferred for mandatory dependencies).
- Setter injection.
- Field injection (@Autowired).

The IoC (Inversion of Control) container manages bean lifecycle and wiring.

Q18

What is JWT Authentication? How did you implement it?**✓ Answer**

JWT (JSON Web Token) is a stateless authentication mechanism.

Flow:

1. User logs in → server validates credentials → generates a signed JWT.
2. Client stores JWT and sends it in Authorization header.
3. Server verifies the token's signature and claims on each request.

In VigilAI MedLink, I implemented JWT using Spring Security + io.jsonwebtoken library to secure REST endpoints between clinic and hospital microservices.

■ **Tip:** Be ready to explain the three parts of JWT: Header, Payload, Signature.

Q19

What are the common HTTP methods and their use in REST APIs?**✓ Answer**

- GET: Retrieve resource (idempotent, no body).
- POST: Create new resource.
- PUT: Replace entire resource.
- PATCH: Partial update.
- DELETE: Remove resource.
- HEAD: GET without response body (used for checks).
- OPTIONS: Describes communication options.

Q20

What is the difference between @Component, @Service, @Repository, and @Controller?**✓ Answer**

All are specializations of @Component (Spring auto-detects them via component scanning).

- @Component: Generic Spring bean.
- @Service: Business logic layer — semantic clarity.
- @Repository: Data access layer — adds exception translation (DataAccessException).
- @Controller: Handles HTTP requests; returns view names.
- @RestController: @Controller + @ResponseBody; returns JSON/XML directly.

Q21

What is @Transactional annotation? When do you use it?**✓ Answer**

@Transactional ensures that a sequence of operations executes within a single database transaction. If any operation fails, the entire transaction is rolled back.

Use it on service methods that perform multiple DB writes that must be atomic. Example: In VigilAI, updating patient triage record and risk score together in one transaction.

Q22

What is WebSocket? How did you use it in VigilAI MedLink?**✓ Answer**

WebSocket provides full-duplex, persistent communication between client and server over a single TCP connection — unlike HTTP which is request-response.

In VigilAI MedLink v2.0, I used Spring WebSocket (STOMP protocol) to push real-time patient triage updates from hospital servers to clinic dashboards without polling. This enabled instant notification when a patient's risk score changed.

Q23

What is Docker and how did you use it in your projects?**✓ Answer**

Docker is a containerization platform that packages an application and all its dependencies into a container — ensuring consistent behavior across environments.

In VigilAI MedLink, I used Docker to:

- Containerize Spring Boot microservices and FastAPI separately.
- Use docker-compose to orchestrate multi-service startup (backend + AI service + DB).
- Ensure the same environment from dev to production.

Benefit: Eliminates 'works on my machine' problems.

Q24

What is the difference between microservices and monolithic architecture?**✓ Answer**

Monolithic: Single deployable unit containing all features; simple to develop initially; harder to scale and maintain as it grows.

Microservices: Application split into small, independently deployable services; each owns its domain and database; communicate via APIs or message queues.

In VigilAI MedLink, I architected separate microservices for clinic management, hospital triage, and AI prediction — allowing each to scale independently.

4 Databases — PostgreSQL & MySQL

Q25

What is the difference between SQL and NoSQL databases?

✓ Answer

SQL (Relational): Structured schema, ACID transactions, uses tables with rows/columns. Best for structured data with complex relationships. Examples: PostgreSQL, MySQL.

NoSQL: Schema-less, horizontally scalable, eventual consistency. Best for unstructured/semi-structured data at scale. Examples: MongoDB, Cassandra, Redis.

In my projects, I chose PostgreSQL because patient records, employee data, and recruitment data are highly relational.

Q26

Explain the different types of SQL JOINS.

✓ Answer

- INNER JOIN: Returns rows where condition is matched in both tables.
- LEFT JOIN: All rows from left table + matched rows from right; NULLs for non-matches.
- RIGHT JOIN: All rows from right table + matched rows from left.
- FULL OUTER JOIN: All rows from both tables; NULLs where no match.
- CROSS JOIN: Cartesian product of both tables.
- SELF JOIN: A table joined with itself (e.g., employee-manager hierarchy).

Q27

What are indexes? Why are they important?

✓ Answer

An index is a data structure (typically B-tree) that speeds up data retrieval at the cost of extra storage and write overhead.

Types:

- Primary Index: On primary key (auto-created).
- Unique Index: Enforces uniqueness.
- Composite Index: On multiple columns.
- Partial Index: On a subset of rows.

In VigilAI MedLink, I indexed `patient_id` and `triage_timestamp` in the `triage records` table to speed up real-time queries during high patient load.

Q28

What are ACID properties?**✓ Answer**

- Atomicity: All operations in a transaction succeed or all are rolled back.
- Consistency: Database moves from one valid state to another.
- Isolation: Concurrent transactions don't interfere with each other.
- Durability: Once committed, data persists even after system failure.

PostgreSQL is fully ACID-compliant, which is why I chose it for healthcare data in VigilAI.

Q29

What is normalization? Explain 1NF, 2NF, 3NF.**✓ Answer**

Normalization organizes a database to reduce redundancy.

- 1NF: Atomic values; no repeating groups.
- 2NF: 1NF + no partial dependency (non-key columns depend on the whole primary key).
- 3NF: 2NF + no transitive dependency (non-key columns depend only on the primary key).

In my Employee Management System, I normalized employee, department, and role into separate tables.

Q30

What is the difference between WHERE and HAVING?**✓ Answer**

WHERE: Filters rows before grouping; cannot use aggregate functions.

HAVING: Filters groups after GROUP BY; can use aggregate functions.

Example: `SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;`

Q31

What is a foreign key and how does it ensure referential integrity?**✓ Answer**

A foreign key is a column in one table that references the primary key of another table.

It ensures referential integrity by:

- Preventing insertion of a row that references a non-existent key.
- Preventing deletion of a referenced row (unless CASCADE is set).

In my EMS project, the employee table has a foreign key to the department table — ensuring no employee can be assigned to a non-existent department.

Q32

How did you design the PostgreSQL schema for VigilAI MedLink?**✓ Answer**

Key tables:

- patients: patient_id (PK), name, dob, clinic_id (FK).
- triage_records: record_id (PK), patient_id (FK), timestamp, symptoms, risk_score.
- hospitals: hospital_id (PK), name, capacity.
- clinic_hospital_link: clinic_id (FK), hospital_id (FK) — many-to-many.

I indexed patient_id and timestamp for fast triage lookups. Risk scores were stored as FLOAT with a separate audit log table for changes.

5

React.js & Frontend Development

Q33

What is React.js and how does it work?

✓ Answer

React is a JavaScript library for building component-based UIs. It uses a Virtual DOM to efficiently update the actual DOM — only re-rendering components whose state or props have changed, rather than the entire page. This makes React highly performant for dynamic applications like dashboards.

Q34

What is the difference between state and props in React?

✓ Answer

Props: Read-only data passed from parent to child component; the child cannot modify props.

State: Mutable data managed within a component; when state changes, the component re-renders.

Example: In my EMS frontend, employee list is fetched and stored in state; each EmployeeCard receives individual employee data as props.

Q35

Explain React Hooks: useState, useEffect, useCallback.

✓ Answer

- useState: Adds local state to a functional component.
- useEffect: Handles side effects (API calls, subscriptions, DOM updates) after render; replaces componentDidMount/Update/Unmount.
- useCallback: Memoizes a function to prevent unnecessary re-creation on every render — useful when passing callbacks to child components.
- useMemo: Memoizes computed values.
- useRef: Accesses DOM elements or persists values without triggering re-renders.

Q36

What is Axios and how did you use it in your projects?

✓ Answer

Axios is a promise-based HTTP client for making API requests from the browser.

In my Employee Management System, I used Axios to:

- GET all employees on component mount (inside useEffect).
- POST new employee data to the Spring Boot API.
- PUT updates and DELETE records.

I configured an Axios instance with a base URL and default headers to avoid repetition across components.

Q37

What is the Virtual DOM? How does React's reconciliation work?**✓ Answer**

The Virtual DOM is a lightweight in-memory representation of the real DOM.

When state changes, React:

1. Creates a new Virtual DOM tree.
2. Diffs it against the previous Virtual DOM (reconciliation).
3. Identifies the minimal set of real DOM operations needed.
4. Applies only those changes (patching).

This is far more efficient than manually manipulating the real DOM on every update.

Q38

How do you handle API errors and loading states in React?**✓ Answer**

I use a pattern with three state variables:

- data: stores fetched result.
- loading: boolean, true while fetching.
- error: stores error message if request fails.

In useEffect, I wrap the Axios call in try-catch; set loading=true before the call and false in finally. Show a spinner when loading, error message if error, and data otherwise.

In VigilAI's frontend, I showed a 'Fetching patient data...' spinner during WebSocket reconnection.

Q39

What is responsive web design? How did you implement it?**✓ Answer**

Responsive design ensures a web app works well on all screen sizes — desktops, tablets, and phones.

Techniques I used:

- CSS Flexbox and Grid for layout.
- Media queries for breakpoint-based styling.
- Relative units (% , rem, em) instead of fixed pixels.
- React's conditional rendering for mobile vs. desktop views.

I also completed the freeCodeCamp Responsive Web Design certification, which deepened my CSS fundamentals.

6

AI / ML — XGBoost, LSTM, NLP, LLMs

Q40

What is XGBoost? Why did you use it for patient risk prediction?**✓ Answer**

XGBoost (Extreme Gradient Boosting) is an optimized gradient boosting algorithm that builds an ensemble of decision trees sequentially, where each tree corrects errors of the previous.

Why for triage?

- Handles tabular/structured data (vitals, symptoms) extremely well.
- Faster training than deep learning for small-to-medium datasets.
- Provides feature importance scores — critical for medical interpretability.
- Won many Kaggle healthcare competitions.

I trained it on patient symptom vectors and vital sign data to output a risk score (0–1) for triage prioritization.

Q41

What is the difference between classification and regression?**✓ Answer**

Classification: Predicts a discrete class label. Example: Is this patient high-risk (Yes/No)? Metrics: Accuracy, Precision, Recall, F1, AUC-ROC.

Regression: Predicts a continuous numeric value. Example: What is this patient's risk score (0.0–1.0)? Metrics: RMSE, MAE, R^2 .

In VigilAI, I used regression output (0–1 score) and then thresholded it for triage classification.

Q42

What is LSTM? Where is it used?**✓ Answer**

LSTM (Long Short-Term Memory) is a type of Recurrent Neural Network (RNN) designed to capture long-range dependencies in sequential data without suffering from the vanishing gradient problem.

Key components: Input gate, Forget gate, Output gate, Cell state.

Applications: Time-series forecasting, NLP (text generation, translation), speech recognition.

In my Datathon 2025 project at REVA University, I used LSTM for sequential patient data prediction — predicting patient deterioration over time from time-series vitals.

Q43

What is NLP? What NLP techniques have you applied?**✓ Answer**

NLP (Natural Language Processing) enables machines to understand and generate human language.

Techniques I've worked with:

- Tokenization: Splitting text into tokens.
- TF-IDF: Term frequency-inverse document frequency for text vectorization.
- Sentiment Analysis: Classifying text sentiment.
- Named Entity Recognition (NER): Extracting entities from text.
- Text classification using pre-trained embeddings.

In the context of VigilAI, NLP can be applied to free-text symptom descriptions to extract structured features for the risk model.

Q44

What are LLMs (Large Language Models)? How do they fit into your stack?**✓ Answer**

LLMs are massive transformer-based models trained on vast text data to understand and generate human language. Examples: GPT-4, Claude, Gemini, Llama.

In my stack, LLMs could:

- Parse unstructured patient notes into structured triage data.
- Power a symptom checker chatbot for clinics.
- Generate automated medical summaries for doctors.
- Enable semantic search over patient records.

I have conceptual knowledge of prompt engineering, RAG (Retrieval-Augmented Generation), and fine-tuning approaches.

■ **Tip:** Mention openness to learning LangChain or similar frameworks.

Q45

How do you evaluate an ML model? What metrics matter most?**✓ Answer**

Depends on the problem type:

Classification: Accuracy (balanced datasets), Precision (minimize false positives), Recall (minimize false negatives), F1 Score (balance precision-recall), AUC-ROC.

Regression: RMSE, MAE, R^2 (coefficient of determination).

For VigilAI's triage model, Recall is most critical — we must not miss high-risk patients (false negatives are more dangerous than false positives in medical triage).

Q46

What is overfitting? How do you prevent it?**✓ Answer**

Overfitting occurs when a model learns the training data too well, including noise, and performs poorly on unseen data.

Prevention techniques:

- Cross-validation (k-fold).
 - Regularization (L1/Lasso, L2/Ridge).
 - Dropout (in neural networks).
 - Early stopping during training.
 - Reducing model complexity.
 - More training data or data augmentation.
 - Feature selection to remove irrelevant features.
-

7 Salesforce — Apex, LWC, SOQL & Flows

Q47 What is Salesforce and what is it used for?

✓ Answer

Salesforce is the world's leading CRM (Customer Relationship Management) platform. It helps businesses manage sales, service, marketing, and custom business processes through a cloud-based environment.

Developers extend it using:

- Apex (Java-like server-side language).
- Lightning Web Components (LWC) for UI.
- SOQL for querying data.
- Flows for declarative automation.

I built a full Recruitment Management System on Salesforce for my Job Recruitment Portal project.

Q48 What is Apex? How is it different from Java?

✓ Answer

Apex is Salesforce's proprietary, strongly-typed, object-oriented language similar to Java but designed specifically for the Force.com platform.

Key differences from Java:

- Runs on Salesforce multi-tenant cloud servers.
- Governor limits enforce fair resource usage (e.g., max 100 SOQL queries per transaction).
- Built-in DML operations (insert, update, delete, upsert).
- Tight integration with Salesforce objects and metadata.
- No file I/O or threading.

Q49 What are Apex Triggers? What are trigger best practices?

✓ Answer

Apex Triggers execute custom code before or after DML events (insert, update, delete, undelete) on Salesforce records.

Trigger events: before insert, after insert, before update, after update, before delete, after delete, after undelete.

Best practices:

- One trigger per object.
- Move logic to handler classes (Trigger Handler Pattern).
- Bulkify triggers to handle 200+ records at once.
- Avoid SOQL or DML inside loops.
- Use trigger context variables (Trigger.new, Trigger.old).

■ **Tip:** Mention the 'one trigger per object' best practice explicitly.

Q50

What is SOQL? How is it different from SQL?**✓ Answer**

SOQL (Salesforce Object Query Language) is similar to SQL but designed to query Salesforce objects.

Differences:

- Only SELECT is supported (no INSERT, UPDATE, DELETE — use DML instead).
- Supports relationship queries (both parent-to-child and child-to-parent).
- Subject to governor limits (max 100 queries per transaction).
- Queries Salesforce objects like Account, Contact, Opportunity.

Example: `SELECT Id, Name, Status__c FROM Application__c WHERE Job__c = :jobId ORDER BY Score__c DESC`

Q51

What are Lightning Web Components (LWC)? How do they differ from Aura Components?**✓ Answer**

LWC are modern Salesforce UI components built using standard web technologies (HTML, JavaScript, CSS) following W3C standards.

Differences from Aura:

- LWC uses standard JS; Aura uses a proprietary framework.
- LWC is faster and lighter.
- LWC uses Shadow DOM for style encapsulation.
- LWC has better performance and is the recommended approach.

In my Recruitment Portal, I built reusable LWC components for candidate profiles, job listings, and application status tracking.

Q52

What are Salesforce Flows? When do you use them instead of Apex?**✓ Answer**

Flows are declarative automation tools in Salesforce that allow you to build business processes without code.

Types: Screen Flows, Auto-launched Flows, Record-Triggered Flows, Scheduled Flows.

Use Flows when:

- Logic is simple and business-user-maintainable.
- No complex loops or dynamic SOQL.
- Quick iteration is needed.

Use Apex when:

- Complex logic, error handling, or external callouts needed.
- Governor limit-sensitive bulk processing.

In my project, I used Record-Triggered Flows to automate status updates when an application was approved.

Q53

What are Salesforce Governor Limits? Why do they exist?**✓ Answer**

Governor limits are runtime limits enforced by Salesforce to ensure no single tenant monopolizes shared resources in the multi-tenant environment.

Key limits (per transaction):

- Max 100 SOQL queries.
- Max 150 DML statements.
- Max 50,000 SOQL rows retrieved.
- Max 10 MB heap size.

To work within limits:

- Bulkify code (process lists, not individual records).
 - Use Maps to avoid repeated queries.
 - Collect DML operations and execute in bulk.
-

8 Data Structures & Algorithms

Q54

What is the time and space complexity of common sorting algorithms?**✓ Answer**

- Bubble Sort: $O(n^2)$ time, $O(1)$ space — rarely used in practice.
- Selection Sort: $O(n^2)$ time, $O(1)$ space.
- Insertion Sort: $O(n^2)$ worst, $O(n)$ best — good for nearly sorted data.
- Merge Sort: $O(n \log n)$ time, $O(n)$ space — stable sort.
- Quick Sort: $O(n \log n)$ avg, $O(n^2)$ worst, $O(\log n)$ space — fastest in practice.
- Heap Sort: $O(n \log n)$ time, $O(1)$ space.
- Counting Sort: $O(n+k)$ — for integers in a known range.

Q55

Explain Binary Search. What are its prerequisites and complexity?**✓ Answer**

Binary search finds a target in a sorted array by repeatedly halving the search space.

Prerequisite: Array must be sorted.

Time: $O(\log n)$ | Space: $O(1)$ iterative, $O(\log n)$ recursive.

Algorithm:

1. Set $low=0$, $high=n-1$.
2. $mid = (low+high)/2$.
3. If $arr[mid]==target$, return mid .
4. If $arr[mid] < target$, $low = mid+1$.
5. Else $high = mid-1$.
6. Repeat until $low > high$.

Q56

What is a Hash Map? How does it handle collisions?**✓ Answer**

A HashMap stores key-value pairs using a hash function to map keys to array indices.

Collision handling:

- Chaining: Each bucket holds a linked list of entries with the same hash.
- Open Addressing (Linear Probing): Find the next empty slot.

Java's HashMap uses chaining; converts linked lists to Red-Black trees when bucket size > 8 (Java 8+). Average $O(1)$ get/put, $O(n)$ worst case.

Q57

Explain Binary Tree vs Binary Search Tree (BST).**✓ Answer**

Binary Tree: Each node has at most 2 children; no ordering constraint.

BST: A binary tree where left child < parent < right child.

BST operations: Search $O(\log n)$ avg, $O(n)$ worst (unbalanced).

Balanced BSTs (AVL, Red-Black Tree): Guarantee $O(\log n)$ for all operations.

In Java, TreeMap uses a Red-Black Tree internally.

Q58

What is BFS vs DFS? When do you use each?**✓ Answer**

BFS (Breadth-First Search): Explores level by level using a Queue. Best for: shortest path in unweighted graphs, level-order traversal.

DFS (Depth-First Search): Explores as deep as possible using a Stack (or recursion). Best for: cycle detection, topological sort, path existence, maze solving.

Both: $O(V+E)$ time, $O(V)$ space for graphs.

Q59

What is Dynamic Programming? Give an example.**✓ Answer**

DP solves problems by breaking them into overlapping subproblems and storing results to avoid redundant computation.

Two approaches:

- Memoization (Top-Down): Recursion + caching.
- Tabulation (Bottom-Up): Iterative DP table.

Classic examples: Fibonacci ($O(n)$ with DP vs $O(2^n)$ naive), LCS, 0/1 Knapsack, Longest Increasing Subsequence.

Key signal: Problem asks for 'minimum', 'maximum', 'number of ways', or 'can you achieve X'.

Q60

What is a stack and queue? Give real-world applications.**✓ Answer**

Stack (LIFO — Last In First Out):

- Applications: function call stack, undo/redo, expression evaluation, browser back button.

Queue (FIFO — First In First Out):

- Applications: BFS traversal, task scheduling, printer queue, message queues (like Kafka).

In Java: Use Deque instead of Stack class; LinkedList or ArrayDeque for Queue.

9

Operating Systems & Computer Networks

Q61

What is a process vs a thread?

✓ Answer

Process: An independent program in execution with its own memory space (heap, stack, code, data).

Thread: A lightweight unit of execution within a process; threads share the same memory space.

Context switching between processes is heavier than between threads. In Java, multithreading (within one JVM process) is common for concurrent request handling in Spring Boot.

Q62

What is deadlock? How do you prevent it?

✓ Answer

Deadlock occurs when two or more threads wait indefinitely for resources held by each other.

Four necessary conditions (Coffman):

1. Mutual Exclusion.
2. Hold and Wait.
3. No Preemption.
4. Circular Wait.

Prevention: Break any one condition. Common approach: acquire locks in a consistent order; use timeouts; avoid nested locks.

Q63

What is the OSI model?

✓ Answer

The OSI (Open Systems Interconnection) model has 7 layers:

1. Physical — bits over wire.
2. Data Link — MAC addresses, frames (Ethernet).
3. Network — IP addressing, routing.
4. Transport — TCP/UDP, ports, segmentation.
5. Session — session management.
6. Presentation — encryption, compression, encoding.
7. Application — HTTP, FTP, DNS, SMTP.

HTTP (used in REST APIs) operates at Layer 7; TCP at Layer 4.

Q64

What is the difference between TCP and UDP?**✓ Answer**

TCP (Transmission Control Protocol): Connection-oriented, reliable delivery, ordered packets, error checking, slower. Used for HTTP, HTTPS, SMTP, FTP.

UDP (User Datagram Protocol): Connectionless, no delivery guarantee, faster, lower overhead. Used for video streaming, DNS, online gaming, VoIP.

Q65

What is HTTP vs HTTPS? What is TLS?**✓ Answer**

HTTP: Hypertext Transfer Protocol — plain text communication; vulnerable to interception.

HTTPS: HTTP + TLS encryption — encrypts data in transit.

TLS (Transport Layer Security): Cryptographic protocol that provides:

- Encryption (symmetric after handshake).
- Authentication (via SSL certificates).
- Integrity (data tampering detection).

All my REST APIs are served via HTTPS; JWT tokens must always be transmitted over HTTPS.

Q66

What is DNS? How does it work?**✓ Answer**

DNS (Domain Name System) translates human-readable domain names (e.g., google.com) into IP addresses.

Resolution process:

1. Browser checks local cache.
2. OS checks /etc/hosts.
3. Query sent to Recursive Resolver (ISP).
4. Resolver queries Root → TLD → Authoritative DNS servers.
5. IP address returned and cached.

Understanding DNS is important when deploying applications on Vercel or cloud platforms.

10 Tools & Platforms — Git, Docker, Vercel, Postman

Q67 What is Git? Explain common Git commands.

✓ Answer

Git is a distributed version control system for tracking code changes and collaborating.

Essential commands:

- `git init` / `git clone` — initialize or copy a repo.
- `git add .` — stage changes.
- `git commit -m 'message'` — record changes.
- `git push` / `git pull` — sync with remote.
- `git branch` / `git checkout -b` — branching.
- `git merge` / `git rebase` — integrate changes.
- `git stash` — temporarily shelve changes.
- `git log --oneline` — view commit history.

All my projects are version-controlled on GitHub.

Q68 What is the difference between `git merge` and `git rebase`?

✓ Answer

`git merge`: Combines two branches, preserving history; creates a merge commit.

`git rebase`: Moves commits from one branch on top of another; rewrites history for a cleaner linear log.

When to use:

- `merge`: For public branches where history should be preserved.
- `rebase`: For feature branches before merging to main (cleaner history).

Q69 How do you use Postman for API testing?

✓ Answer

Postman is a GUI tool for building, testing, and documenting REST APIs.

I use it to:

- Send GET, POST, PUT, DELETE requests to my Spring Boot endpoints.
- Set headers (e.g., Authorization: Bearer).
- Inspect response bodies, status codes, and timings.
- Create collections to organize tests per project.
- Write pre-request scripts and tests to automate API validation.

For VigilAI and EMS, I used Postman extensively before connecting the React frontend.

Q70

What is Vercel and how did you deploy projects on it?**✓ Answer**

Vercel is a cloud platform optimized for frontend deployment, especially React/Next.js apps.

Deployment workflow:

1. Connect GitHub repo to Vercel.
2. Vercel detects React app (Create React App / Vite).
3. Builds and deploys automatically on every push to main branch.
4. Provides a global CDN, HTTPS by default, and preview deployments for PRs.

I deployed my React.js frontends on Vercel for fast, zero-config hosting.

11 Project Deep Dive — VigilAI MedLink v2.0

Q71 Explain VigilAI MedLink v2.0 end-to-end.

✓ Answer

VigilAI MedLink is a real-time healthcare triage platform connecting clinics and hospitals.

Architecture:

- Frontend: React.js dashboard for clinics and hospitals.
- Backend: Spring Boot microservices for clinic management, hospital management, and authentication.
- AI Service: FastAPI (Python) hosting the XGBoost risk prediction model.
- Database: PostgreSQL for patient, triage, and hospital data.
- Communication: REST APIs + WebSocket for real-time updates.
- Security: JWT authentication for all service-to-service and client-to-server communication.
- Deployment: Docker + docker-compose.

Flow: Clinic registers a patient → symptoms submitted → XGBoost model scores risk → score pushed via WebSocket to hospital dashboard → triage nurse acts accordingly.

Q72 What was the biggest technical challenge in VigilAI and how did you solve it?

✓ Answer

Challenge: Integrating real-time data push (WebSocket) across microservices while maintaining JWT security.

Problem: WebSocket connections need authentication but don't support standard HTTP Authorization headers during the handshake.

Solution: I passed the JWT as a query parameter during the WebSocket handshake, validated it server-side in a custom Spring Security WebSocket interceptor, and set up STOMP message-level authorization for subsequent messages.

Lesson: Security in real-time systems requires extra design consideration beyond standard REST patterns.

Q73 Why did you use FastAPI for the AI service instead of Spring Boot?

✓ Answer

- Python's ML ecosystem (scikit-learn, XGBoost, pandas, numpy) is vastly richer than Java's.
- FastAPI is async-native, fast, and has automatic OpenAPI docs generation.
- Keeping AI logic in Python allows data scientists to own that service without learning Java.
- The microservice communicates with Spring Boot via REST APIs, so language doesn't matter at the service boundary.

This taught me that polyglot microservices are a valid and practical architecture.

Q74

How did you ensure data integrity in patient triage records?**✓ Answer**

- Used PostgreSQL transactions (@Transactional in Spring) to ensure atomic updates to triage records and risk scores.
 - Added NOT NULL constraints and CHECK constraints in the schema (e.g., risk_score BETWEEN 0 AND 1).
 - Implemented optimistic locking with @Version in JPA entities to prevent lost updates under concurrent edits.
 - Stored an audit log of all triage record changes with timestamps and user IDs for accountability.
-

12 Certifications & Achievements

Q75

Tell me about your Oracle Databases for Developers Foundations certification.

✓ Answer

This Oracle certification covered:

- Relational database design principles.
- SQL fundamentals: SELECT, JOIN, aggregation, subqueries.
- Database objects: tables, views, indexes, sequences.
- PL/SQL basics: procedures and functions.
- Transaction management and ACID properties.

It solidified my SQL foundation before I started designing PostgreSQL schemas for my projects.

Q76

What did you learn from the Cloud Computing certification (Infosys Springboard)?

✓ Answer

The certification covered:

- Cloud service models: IaaS, PaaS, SaaS.
- Deployment models: public, private, hybrid cloud.
- Core concepts: virtualization, elasticity, on-demand provisioning.
- Introduction to major cloud providers: AWS, Azure, GCP.

This gave me the conceptual foundation to understand how Docker containers and Vercel deployments fit into cloud infrastructure.

Q77

Tell me about winning Datathon 2025 at REVA University.

✓ Answer

Datathon 2025 was a competitive data science challenge hosted by REVA University where I competed against teams across the university.

I built a machine learning solution that involved:

- Data preprocessing and feature engineering on a healthcare/tabular dataset.
- Model selection and training (applied LSTM and XGBoost).
- Result interpretation and visualization.

Winning validated my ability to apply ML techniques under time pressure and communicate insights effectively — skills directly applicable to AI-powered product development.

■ **Tip:** This is a great differentiator — emphasize it in every interview.



Soft Skills — Most Asked Interview Questions

Q1**Tell me about yourself (fresher perspective).**

Start with your education, transition to your technical skills and projects, and close with your career goal. Keep it to 90 seconds.

Example: 'I'm a CSE student at REVA University with a strong foundation in full-stack development. I've built projects like VigilAI MedLink — an AI-powered healthcare triage system — and won the Datathon 2025. I'm passionate about building scalable, AI-integrated software and am excited to bring this to a professional setting.'

Q2**What are your strengths?**

Choose 2–3 genuine strengths with evidence.

Example: 'My top strength is problem-solving — I demonstrated this by architecting a real-time microservices system for VigilAI MedLink and winning the REVA Datathon 2025. I'm also highly adaptable — I taught myself FastAPI and Docker to complete that project. Lastly, I communicate complex technical ideas clearly, which helped me document and present my Salesforce Recruitment Portal.'

Q3**What is your greatest weakness?**

Be honest but show self-awareness and a plan.

Example: 'I sometimes get too focused on perfecting the technical implementation and lose sight of the deadline. I've been working on this by setting time-boxed milestones for each feature and doing daily check-ins against my project plan. This helped me deliver VigilAI MedLink on schedule despite its complexity.'

Q4**Why should we hire you?**

'You should hire me because I bring a rare combination — full-stack development (React + Spring Boot), database design, Salesforce CRM development, and AI/ML integration. I don't just know theories; I've built and deployed real projects that solve real problems. I'm a fast learner, a proven winner (Datathon 2025), and genuinely passionate about this domain.'

Q5**Describe a time you faced a challenge and how you overcame it.**

Use the STAR method.

Situation: Integrating WebSocket with JWT security in VigilAI.

Task: Ensure real-time updates are authenticated.

Action: Researched Spring Security WebSocket interceptors, tested multiple approaches, implemented token validation at the handshake level.

Result: Secure, real-time data flow across microservices.

Lesson: Security in real-time systems requires proactive design, not afterthought.

Q6

How do you handle working under pressure or tight deadlines?

I break the problem into smaller tasks and prioritize ruthlessly. During the Datathon 2025, I had 24 hours to build a complete ML solution. I created a quick plan: 1 hour for data exploration, 2 hours for feature engineering, 3 hours for model training and tuning, 2 hours for visualization and presentation. This structure kept me on track and I delivered a winning solution. I stay calm by focusing on what I can control.

Q7

Tell me about a time you worked in a team.

In my academic coursework and hackathon experiences, I've collaborated on group projects. I typically take ownership of backend or data pipeline components while coordinating with teammates on frontend or presentation. I believe in clear ownership, regular sync-ups, and honest communication about blockers. Strong collaboration is why I structure my GitHub repos with clear README files and commit messages.

Q8

Where do you see yourself in 5 years?

In 5 years, I see myself as a Senior Software Engineer or Tech Lead, with production experience in distributed systems and applied AI. I want to have shipped products used by real users at scale, mentored junior developers, and contributed to open-source projects. I'm genuinely excited about the convergence of AI and software engineering — that's the space I want to build expertise in.

Q9

Why do you want to work at this company?

Research the company first. Then:

'I'm drawn to [Company] because of [specific product/mission/tech stack]. Your work in [domain] aligns directly with my passion for [area]. I want to contribute to solving [specific problem] with the skills I've built in React, Spring Boot, and AI/ML. The opportunity to learn from experienced engineers here while shipping real products is exactly what I'm looking for at this stage of my career.'

Q10

How do you keep your technical skills up to date?

I learn by building. Whenever I encounter a new technology, I immediately start a small project with it — that's how I learned FastAPI, Docker, and SOQL. I also follow engineering blogs, watch system design videos, and practice DSA regularly. My certifications (Oracle, Infosys, Simplilearn, freeCodeCamp) reflect deliberate structured learning. And I treat every project as a chance to explore one new technology I haven't used before.

Q11

Describe your ideal work environment.

I thrive in environments where there's a strong learning culture, clear ownership of work, and a team that values both technical excellence and collaboration. I'm equally comfortable working independently (I've built full projects solo) and collaborating with a team. I prefer environments where code quality matters, feedback is constructive, and innovation is encouraged.

Q12

How do you prioritize tasks when you have multiple deadlines?

I use a simple framework: assess urgency and impact. High-impact, high-urgency tasks go first. I break them into smaller sub-tasks with time estimates. I use tools like Notion or GitHub Projects to track progress. For example, during the Datathon, I time-boxed each phase explicitly. For coursework, I track deadlines in a weekly planner and start complex tasks at least 3 days early.

Q13

Tell me about a time you received critical feedback.

During a peer code review, a classmate pointed out that my VigilAI API endpoints lacked proper input validation — any malformed request would cause an unhandled exception. Instead of being defensive, I thanked them, studied Spring Validation annotations (`@Valid`, `@NotNull`, `@Size`), and refactored all endpoints. I also added global exception handling with `@ControllerAdvice`. The feedback made my project significantly more robust.

Q14

What motivates you as a developer?

Seeing something I built work in the real world. When I got the XGBoost model to correctly predict patient risk scores and saw the score appear in real-time on the hospital dashboard via WebSocket — that feeling is what drives me. I'm also motivated by learning: every new project teaches me something new. And I genuinely enjoy the problem-solving process — debugging a complex issue and finally finding the root cause is deeply satisfying.

Pro Tips for Every Interview

- Use the **STAR method** (Situation · Task · Action · Result) for all behavioral questions.
- Always tie your answers back to your real projects — VigilAI, EMS, Salesforce Portal.
- Keep GitHub repos clean with good READMEs before every interview.
- Review your VigilAI and EMS codebases so you can speak to every design decision confidently.